# Process & Decision Documentation

## Project/Assignment Decisions

- I initially attempted to build the debug system using separate module files (debuoverlay.js, debugmenu.js, debustate.js)  with import/export, but this caused the game to fail due to comparability issues with live server, p5 and p5play, I simplified the approach by consolidating everything into a single debug.js file using global variables.
- I also added a debugpaused condition tied to debugstate.menuopen so that gameplay pauses when the debug menu is open, it made testing more controlled

## Side Quests and A4 (Individual Work)

- To avoid disrupting the game structure, I put debg featured in a single debugstate object rather than modifying multiple systems
- Features like invincibility, collision boxes, and win condition overrides were implemented as toggles, debug visuals were added as optional overlays to preserve normal game experience

## Role-Based Process Evidence

```
draw(game) {
  if (!debugState.overlayOpen) return;

  camera.off();
  push();

  noStroke();
  fill(0, 180);
  rect(6, 6, 230, 110, 8);

  fill(255);
  textAlign(LEFT, TOP);
  textSize(11);

  const lines = [
    `Score: ${game.score}`,
    `Health: ${game.health}/${game.maxHealth}`,
    `Player: ${Math.round(game.playerX)}, ${Math.round(game.playerY)}`,
    `Grounded: ${game.grounded}`,
    `Dead: ${game.dead}  Won: ${game.won}`,
    `Invuln: ${game.invulnTimer}`,
```

```
  // display a death or win overlay if those events happen
  if (dead) drawDeathOverlay();
  if (won) drawWinOverlay();

  camera.off();

  push();
  textFont("monospace");
  fill("white");
  strokeWeight(2);

  stroke("black");      You, 3 minutes ago • Uncommitted changes
  textSize(9);
  textAlign(LEFT, TOP);
  text('PRESS "`" TO OPEN DEBUG', 6, 18);
  pop();

  camera.on();
```

- Identified module compatibility issues
- Integrated debug toggles into gameplay logic
- Tested featjures iteratively in oslation

### *GenAI Documentation*

GenAi was used for this project.

**Date Used**: March 22, 2026

**Tool Disclosure**: ChatGPT (GPT-5.3)

**Purpose of Use**: Use to debug integration issues and structure the debug system.

**Summary of Interaction**: Helped with organizing debug components (menum overlay, state), resolving module issues, and implementing toggles and pause logic.

**Human Decision Point(s)**:  I choae to abandon the module approach, integrated toggles into existing functions, and refine the UI layout based on testing.

**Integrity & Verification Note**:

Features were verified through playtesting. Tested each change and reverted to stable versions when needed, snruing the game remained functional.

**Scope of GenAI Use**:

Genai supported structured and debugging guidance

**Limitations or Misfires**:

GenAI occasionally suggested incompatible approaches, requiring me to simplify and validate all implementations through testing.

*Summary of Process (Human + Tool)*

I built the debug system gradually and used example 1 from this week's examples to guide me in (UI → toggles → gameplay integration) simplifying when issues arose.

*Decision Points & Trade-offs*

- I sacrifice modularity (seperating files) and adding module in script for global to be more compatible. I added pause during debug, I added logic but improved usability.

*Verification & Judgement*

I playtested the code each time along the way ensuring the game does not break that way it  is easier to retrace my steps to debug. I cross-checked with this week's example to see if the debug menu behaves similar to the example.

*Limitations, Dead Ends, or Open Questions*

- Module implemented failed due to compatibility issues, UI required iteration, and some debug features simplified to maintain stability.

# Appendix

## Prompt:

I need to create a debug screen similar to this example. It uses separate files like `debugoverlay.js` and `debugmenu.js`, but I'm doing everything in `sketch.js`. How should I integrate it?

```
export class DebugOverlay {
 constructor() {
   this.enabled = false;
   this.lines = [];
   this.maxLines = 8;
 }

 toggle() {
   this.enabled = !this.enabled;
 }

 draw({ game } = {}) {
   if (!this.enabled) return;

   fill(255);
   text(`Score: ${game.score}`, 12, 22);
 }
}
```

## Response:

Using `import/export` with p5 and Live Server can prevent the game from loading. A simpler approach is to remove modules and use a global debug file.

```
// debug.js
const debugState = {
 menuOpen: false,
 overlayOpen: false,
 collisionBoxes: false,
 playerInvincible: false,
 winScoreOne: false,
};
<script src="debug.js"></script>
<script src="sketch.js"></script>
```

## Prompt:

The debug menu still isn't showing up.

## Response:

The menu needs to be toggled with input and drawn inside the `draw()` loop.

```
function keyPressed() {
 if (key === "`") {
   debugMenu.toggle();
   return false;
 }
}
function draw() {
 debugMenu.draw();
}
```

## Prompt:

The debug toggles like collision boxes don’t actually do anything.

## Response:

The debug state must be connected to rendering logic.

```
function drawCollisionDebug() {
 if (!debugState.collisionBoxes) return;

 stroke("green");
 noFill();
 rect(player.x - player.w/2, player.y - player.h/2, player.w, player.h);
}
```

## Prompt:

Some debug options like invincibility and win condition aren’t working.

## Response:

Integrate debug state into gameplay conditions.

```
if (debugState.playerInvincible || invulnTimer > 0) return;
const targetWinScore = debugState.winScoreOne ? 1 : WIN_SCORE;
```

## Prompt:

I want to add “press ` to open debug” under the HUD.

## Response:

You can draw it using the HUD function:

```
drawOutlinedTextToGfx(hudGfx, 'press "`" to open debug', 6, 18, "#ffffff");
```

## Prompt:

That looks blurry and pixelated.

## Response:

Draw the text directly on the canvas instead of using `hudGfx`.

```
camera.off();

fill(255);
textSize(10);
text('press ` to open debug', 6, 24);

camera.on();
```

## Prompt:

The text is still hard to read.

## Response:

Add a background and increase contrast.

```
fill(0, 180);
rect(4, 20, 200, 18, 6);

fill(255);
textSize(11);
text('press ` to open debug', 8, 23);
```

## Prompt:

The font itself is hard to read.

## Response:

Use a system font to improve clarity.

```
textFont('16px monospace');
```

## Prompt:

Can we also shift the debug menu left because it’s going off screen?

## Response:

Adjust the menu position and text alignment.

```
rect(10, 22, 240, 140, 14);
text("DEBUG MENU", 30, 40);
text(`${opt.label}: ${value}`, 20, y);
```

## Prompt:

I want the debug hint text smaller without changing my font system.

## Response:

Use standard `text()` with a smaller size instead of modifying your bitmap font.

```
textSize(9);
fill(180);
text('press ` to open debug', 6, 18);
```